

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	SRIVATSAN L	Reg. No.:	192521480
Section / Batch:	2nd	Date:	17/07/2026

Code of Conduct

I SRIVATSAN L (192521480) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: SRIVATSAN L

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

Annexure A – Analytical Problem Solving

S.No	REG NO	NAME	ANALYTICAL PROBLEM SOLVING
------	--------	------	----------------------------

36	192521480	SRIVATSAN L	1. Given a singly linked list L: $L1 \rightarrow L2 \rightarrow L3 \dots \rightarrow L_{n-1} \rightarrow L_n$, reorder and print it to: $L1 \rightarrow L_n \rightarrow L2 \rightarrow L_{n-1}$. 2. Insert the following elements 150, 250, 350, 450, 550 into the double linked list and delete the elements 150 and 250 from the double linked list. Show the linked structure using a node diagram before and after delete operations.
----	-----------	-------------	---

1.Problem Statement

We are given a singly linked list L represented as $L1 \rightarrow L2 \rightarrow L3 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$. The list must be reordered in place, without using extra node storage (such as an array or a second list of copied values) and ideally without altering the data inside the nodes — only the links (next pointers) may be changed. The required output ordering interleaves the list from both ends towards the centre:

Required transformation

$L1, L2, L3, \dots, L_{n-1}, L_n \rightarrow L1, L_n, L2, L_{n-1}, L3, L_{n-2}, \dots$

In other words, the first node of the output is the original head, the second node is the original tail, the third node is the second node from the front, the fourth node is the second node from the back, and so on — the two ends are “zipped” together until they meet in the middle.

2. Key Idea / Strategy

A singly linked list only allows forward traversal, so a node’s predecessor is unknown and the list cannot be walked backward directly. Repeatedly scanning from the head to find the current “last unused” node for every pair would cost $O(n)$ per pair and $O(n^2)$ overall. We avoid this by transforming the problem into three simpler, well-known sub-problems, each solvable in $O(n)$ time and $O(1)$ extra space:

1. Find the middle of the list using the slow/fast (tortoise-and-hare) pointer technique, splitting it into a first half and a second half.
2. Reverse the second half in place, so it can now be walked from L_n backwards (i.e., $L_n \rightarrow L_{n-1} \rightarrow L_{n-2} \rightarrow \dots$) using only next pointers.
3. Merge the first half and the reversed second half by alternately picking one node from each list.

Because each of the three phases is a single linear pass, the combined algorithm runs in $O(n)$ time using only a constant number of pointer variables — satisfying the in-place, no-extra-list requirement.

4. Diagrammatic Walkthrough (Node Diagrams)

Consider the example list with $n = 6$ nodes: $L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow L_4 \rightarrow L_5 \rightarrow L_6 \rightarrow \text{NULL}$. Each box below represents one node, split into a data field and a next-pointer field, exactly as it would be represented in memory.

Figure 1: The original singly linked list before any reordering.

Figure 2: slow advances one node at a time while fast advances two; when fast reaches the end, slow is at the middle (L3), marking the end of the first half.

Figure 3: The list is split into two independent halves by setting L3.next = NULL.

Figure 4: The second half is reversed in place, so its head pointer now refers to the original tail, L6.

Figure 5: The two halves are merged by alternately taking one node from each, producing the required order L1 → L6 → L2 → L5 → L3 → L4 → NULL.

5. Algorithm (Step-by-step)

Step A — Find the middle

Slow = head, fast = head

While (fast != NULL AND fast.next != NULL):

 Slow = slow.next

 Fast = fast.next.next

// 'slow' now points to the last node of the first half

Step B — Split and reverse the second half

Second = slow.next

Slow.next = NULL // terminate first half

Prev = NULL

Curr = second

While (curr != NULL):

 nextTemp = curr.next

 curr.next = prev

 prev = curr

 curr = nextTemp

second = prev // 'second' now points to Ln

Step C — Merge the two halves alternately

First = head

While (second != NULL):

 Temp1 = first.next

 Temp2 = second.next

```
First.next = second
```

```
If (temp1 != NULL):
```

```
    Second.next = temp1
```

```
First = temp1
```

```
Second = temp2
```

6. Complete C Implementation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
Struct Node {
```

```
    Int data;
```

```
    Struct Node* next;
```

```
};
```

```
// Insert a new node with given value at the end of the list
```

```
Struct Node* insertEnd(struct Node* head, int value) {

    Struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;

    newNode->next = NULL;

    if (head == NULL) return newNode;

    struct Node* temp = head;

    while (temp->next != NULL) temp = temp->next;

    temp->next = newNode;

    return head;

}
```

// Reverse a singly linked list iteratively

```
Struct Node* reverseList(struct Node* head) {

    Struct Node *prev = NULL, *curr = head, *nextTemp;

    While (curr != NULL) {

        nextTemp = curr->next;
```

```
curr->next = prev;
```

```
prev = curr;
```

```
curr = nextTemp;
```

```
}
```

```
Return prev;
```

```
}
```

```
// Reorder list: L1 L2 ... Ln -> L1 Ln L2 Ln-1 ...
```

```
Void reorderList(struct Node** headRef) {
```

```
    If (*headRef == NULL || (*headRef)->next == NULL) return;
```

```
// Step A: find the middle with slow/fast pointers
```

```
Struct Node *slow = *headRef, *fast = *headRef;
```

```
While (fast != NULL && fast->next != NULL) {
```

```
    Slow = slow->next;
```

```
    Fast = fast->next->next;
```



```
}
```

```
// Step B: split into two halves and reverse the second half
```

```
Struct Node* second = slow->next;
```

```
Slow->next = NULL;
```

```
Second = reverseList(second);
```

```
// Step C: merge the two halves alternately
```

```
Struct Node* first = *headRef;
```

```
While (second != NULL) {
```

```
    Struct Node* temp1 = first->next;
```

```
    Struct Node* temp2 = second->next;
```

```
    First->next = second;
```

```
    If (temp1 != NULL) second->next = temp1;
```

```
    First = temp1;
```

```
    Second = temp2;
```

```
}
```

```
}
```

```
Void printList(struct Node* head) {
```

```
While (head != NULL) {
```

```
Printf("L%d", head->data);
```

```
If (head->next != NULL) printf(" -> ");
```

```
Head = head->next;
```

```
}
```

```
Printf("-> NULL\n");
```

```
}
```

```
Int main() {
```

```
Struct Node* head = NULL;
```

```
For (int i = 1; i <= 6; i++) head = insertEnd(head, i);
```

```
Printf("Original list : ");
```

```
printList(head);
```

```
reorderList(&head);
```

```
printf("Reordered list: ");
```

```
printList(head);
```

```
return 0;
```

```
}
```

The program above allocates the list dynamically, applies reorderList(), and prints both the original and reordered sequences.

7. Dry Run / Sample Execution

Tracing the algorithm on the six-node example list makes each phase concrete:

Stage	Resulting State
-------	-----------------

Input list L	L1 → L2 → L3 → L4 → L5 → L6 → NULL
--------------	------------------------------------

N (length)	6
------------	---

Middle found (slow) L3 (first half ends here)

First half L1 → L2 → L3 → NULL

Second half L4 → L5 → L6 → NULL

Second half reversed L6 → L5 → L4 → NULL

Alternate merge take L1, then L6, then L2, then L5, then L3, then L4

Final output L1 → L6 → L2 → L5 → L3 → L4 → NULL

Program Output

Original list : L1 -> L2 -> L3 -> L4 -> L5 -> L6 -> NULL

Reordered list: L1 -> L6 -> L2 -> L5 -> L3 -> L4 -> NULL

8. Complexity Analysis

Operation	Time Complexity	Space Complexity
-----------	-----------------	------------------

Find middle node (slow/fast pointer)	O(n)	O(1)
--------------------------------------	------	------

Reverse second half	O(n)	O(1)
---------------------	------	------

Merge two halves alternately	O(n)	O(1)
------------------------------	------	------

Overall Algorithm	O(n)	O(1)
-------------------	------	------

Each of the three phases — finding the middle, reversing the second half, and merging — makes a single pass over at most n nodes, so the total time is $O(n)$. Only a fixed number of pointer variables (slow, fast, first, second, prev, curr, temp1, temp2) are used regardless of n , giving $O(1)$ auxiliary space. No new nodes are created and no array is used to store node references, satisfying an in-place reordering.

9. Edge Cases Considered

- Empty list (head = NULL): the function returns immediately — nothing to reorder.
- Single node list: head->next is NULL, so the function returns the list unchanged.
- Two-node list ($L1 \rightarrow L2$): the middle pointer stops at $L1$, the second half is just $L2$, and merging yields $L1 \rightarrow L2$, which is already the correct (and only possible) reordering.
- Odd number of nodes (e.g., $n = 5$): the slow pointer naturally lands on the exact middle node, which is correctly appended by the merge loop when temp1 becomes NULL for the last node.

10. Conclusion

The reordering problem $L1, L2, \dots, L_n \rightarrow L1, L_n, L2, L_{n-1}, \dots$ is efficiently solved by decomposing it into three classic singly-linked-list operations — middle-finding, in-place reversal, and alternate merging — each running in linear time. This avoids the $O(n^2)$ cost of repeated end-to-end traversal and uses only $O(1)$ extra space, making it an optimal in-place solution suitable for lists of any length, including both even and odd node counts.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity

Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, wellstructured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear
Faculty In-charge		Course Coordinator		Program Director	
Signature: _____		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	